

Scoping Brief: Real-Time Crypto Volatility Detection

Fundamentals of Operationalizing AI — Individual Assignment

1. Use Case

Cryptocurrency markets are highly volatile and operate 24/7. Traders and risk managers need early warning signals when volatility is about to spike so they can adjust positions, tighten stop-losses, or pause automated strategies. This project builds a real-time pipeline that ingests live BTC-USD ticker data from Coinbase, engineers features from the streaming data, and uses a trained model to flag imminent volatility spikes within the next 60 seconds.

2. 60-Second Prediction Goal

The system predicts whether a **volatility spike** will occur in the next 60 seconds. Specifically:

Component	Definition
Target horizon	60 seconds into the future
Volatility proxy	Rolling standard deviation of midprice returns over the next 60s window
Label definition	1 if $\sigma_{\text{future}} \geq \tau$ (threshold); else 0
Threshold τ	To be determined via EDA (percentile analysis of observed volatility)

The midprice is computed as $(\text{best_bid} + \text{best_ask}) / 2$ from live Coinbase ticker data. Returns are computed as the percentage change between consecutive midprice observations.

3. Success Metric

The primary evaluation metric is **PR-AUC (Precision-Recall Area Under the Curve)**. This is chosen because volatility spikes are rare events (class imbalance), making accuracy and ROC-AUC misleading. PR-AUC focuses on how well the model identifies the minority class (spikes) without excessive false alarms.

Secondary metric: **F1 score at an optimized threshold**, to assess operational performance at a specific decision boundary.

Metric	Target
PR-AUC	> 0.15 (meaningful above random baseline)

F1@threshold	> 0.20 (usable for alerting)
--------------	------------------------------

4. Risk Assumptions

Risk	Likelihood	Impact	Mitigation
WebSocket disconnects during data collection	Medium	Medium	Exponential backoff reconnect, heartbeat monitoring, local NDJSON mirror
Low-volatility period during data collection	Medium	High	Collect data across multiple sessions; use replay for reproducibility
Class imbalance (few spike labels)	High	High	Use PR-AUC metric; tune threshold via percentile analysis in EDA
Feature leakage from future data	Medium	Critical	Strict time-based splits; replay script verifies consistency
Kafka message loss or duplication	Low	Medium	acks=all on producer; consumer check script validates counts
Model overfitting on short data window	Medium	High	Simple baseline comparison; regularization; cross-validation

5. Data & Pipeline Summary

Data source: Coinbase Advanced Trade WebSocket API (public ticker channel)

Trading pair: BTC-USD

Pipeline flow: Coinbase WS → ws_ingest.py → Kafka (ticks.raw) → featurizer.py → Kafka (ticks.features) + Parquet → train.py → MLflow

Infrastructure: Docker Compose (Kafka KRaft + MLflow), Python 3.13

Storage formats: NDJSON (raw), Parquet (processed features)